Technical Notes: RAG-based Financial Report Q&A Chatbot

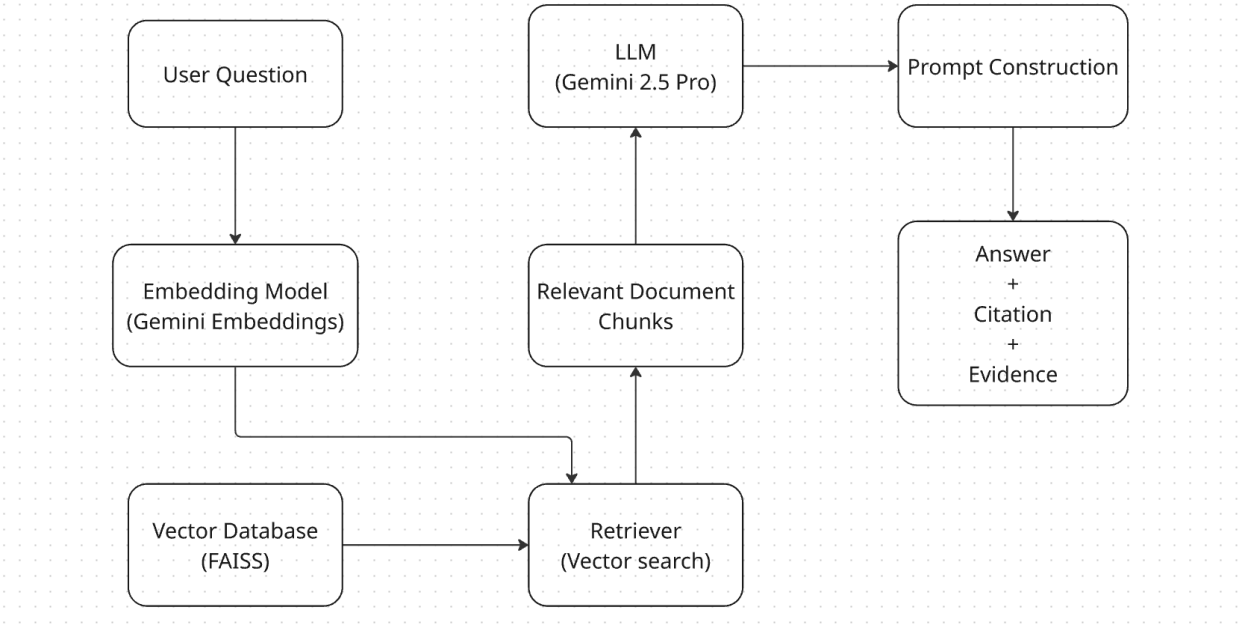
1. System Architecture

1.1 Overview

The system implements a Retrieval-Augmented Generation (RAG) pipeline to enable users to ask questions about the 10-K financial reports of Alphabet, Amazon, and Microsoft. These reports contain detailed information about financial performance, business segments, risk factors, and strategic outlook. The chatbot retrieves relevant sections from uploaded PDF documents and generates answers using Gemini as the large language model. The interface is built using Streamlit, allowing users to parse uploaded documents, select document scope, and interact with the chatbot in a conversational interface. The system was designed to support: Multi-document retrieval, Multi-turn conversation, Selectable retrieval scope, and Accessible citation evidence.

1.2 RAG Pipeline

The overall pipeline of the system is illustrated below:



1.3 System Prompt and Persona

The chatbot uses a system prompt that defines its behavior, constraints, and identity. This system prompt is not visible to the user and is sent to the chatbot with every user prompt.

Persona Prompt:

```
You are a precise, helpful assistant that answers questions based on
the provided documents.

Answer the user's question using only information from the documents.
Scale the depth of your answer to the complexity of the question —
concise for simple questions, thorough for complex ones. If the
documents only partially answer the question, provide what you can
and clearly note what is not covered.

Structure your response for readability. For example, use paragraphs
for explanations, bullet points for lists or comparisons, and headers
only when the answer has multiple parts.

Cite every factual claim inline using bracket numbers like [1], [2].
If multiple sources support a claim, cite all of them. If the answer
cannot be found in the documents at all, respond: "I don't have
enough information to answer this question."
```

The persona prompt informs the chatbot of its role, output structuring and aesthetics, citation formatting, and how to handle impossible questions.

The bot must retrieve its answers only from the contents of the uploaded documents. While the persona stresses complete answers, it also has systematic methods for addressing edge cases.

If the bot is only able to generate a partial answer, it will provide the partial answer and note missing context. If given a question requiring external context, it says “I don’t have enough information to answer this question.”. This programmed response helps to prevent hallucinations and out-of-context answers.

Answer length and depth is scaled to the complexity of the question. Simple questions will receive shorter, more concise answers while more complex questions will receive more detailed and thorough answers. The bot tailors its response to the user's needs by providing precise responses.

Finally, the persona dictates output citations, aesthetics, and structuring. Inline citations will be implemented for all factual claims. Structuring techniques like multiple paragraphs, bullet points, and headers are given as example aesthetics strategies. It is vital that these are explicitly labeled as examples, as this allows the bot to explore alternate output structuring techniques and be creative with its presentation of data.

In summary, the persona prompt of the bot is key in how the bot structures its answers by providing examples, specific edge case responses, and clear behavior guidelines. It enhances the bot's precision by streamlining length, depth, format, and citation-related decision-making.

2. Model Choices

2.1 Large Language Model (LLM)

We used **Gemini 2.5 Pro**, here are some reasons why we chose it.

First, Gemini 2.5 Pro features an impressive long-context capability, enabling it to handle a context window of up to 1 million tokens. This allows it to handle multiple 10-K reports or very large single reports efficiently, ensuring that critical contextual details are not missed, even across thousands of pages.

Example: When a user asks a question like: "Compare the three companies' approaches to artificial intelligence as described in their 2024 10-Ks".
Gemini 2.5 Pro can locate, compare, and synthesize relevant data from various sections in 3 articles, such as the balance sheet, income statements and footnotes.

Second, Gemini 2.5 Pro demonstrates high proficiency in legal reasoning, particularly when working with long-form inputs, ensuring accurate comprehension and analysis of complex legal language and documents without losing context.

Example: When a user asks a question like: "Based on the 2024 10-K filings, which company (Alphabet, Amazon, or Microsoft) faces the most significant and direct legal and regulatory threat to its core business model, and what is the specific evidence to support this claim? Consider the nature of the threat, potential financial impact, and the extent to which it could force a change in business operations."
Gemini 2.5 Pro could process all 3 documents simultaneously, searching for legal and regulatory risks that may be

scattered across various sections of each filing, without losing context. Gemini 2.5 Pro ensures that it connects relevant details from different sections, so that no critical information is overlooked or misinterpreted.

Finally, Gemini 2.5 Pro is natively supported through `langchain_google_genai`, which simplifies integration into RAG pipelines and enhances workflow automation. Developers can quickly set up RAG-based analysis workflows to fetch, retrieve, and synthesize data from multiple 10K filings, making it easier to scale analysis across hundreds of companies.

2.2 Embedding Model

The embedding model we used is `gemini-embedding-001`. The purpose is to convert text chunks into vector embeddings and enable semantic retrieval. The reason for choosing `gemini-embedding-001`, `temperature=0.5(for moderate determinism)` is that it's compatible with the Gemini ecosystem, which provides good semantic performance and easy integration with LangChain, and a good quality for long-document comprehension and retrieval-based QA.

The observed limitation is Gemini embedding API quota limits (HTTP 429) when processing large PDF files. This occurred during the embedding step in `FAISS.from_documents()` when many chunks were embedded in a short period.

The error is shown like this:

```
langchain_google_genai._common.GoogleGenerativeAIError:
Error embedding content: 429 You exceeded your current quota
Quota exceeded for metric:
generativelanguage.googleapis.com/embed_content_paid_tier_reques
ts
limit: 3000
model: gemini-embedding-1.0
Please retry in 28s
```

To address this issue, we implemented several measures:

1. Added retries with exponential backoff to handle transient errors more robustly;
2. Enabled disk caching of FAISS indexes to avoid redundant re-embedding operations;

3. Set a maximum limit on the number of chunks processed per file to optimize resource usage and prevent excessive memory consumption.

2.3 Insights from Model Exploration

After evaluating different components of the system, we found that while alternative language models could be considered as potential choices (e.g. GPT-family models, and Claude), the retrieval strategy and overall system configuration had a more significant impact on the final performance.

In particular, factors such as chunk size, chunk overlap, top-k retrieval, and document preprocessing had a direct impact on whether the model received the correct context. When the retrieved context was incomplete or irrelevant, even strong language models could produce inaccurate or hallucinated responses. Conversely, when the retrieval pipeline successfully provided the relevant document sections, the generation model was generally able to produce accurate and well-grounded answers.

For this reason, our work placed greater emphasis on optimizing the retrieval pipeline and system configuration rather than focusing solely on changing the underlying language model.

3. RAG Configuration

3.1 Key RAG Parameters

Parameters' roles in the RAG are easy to understand by their names, so I'll just paste context or explain a sentence for each.

- MAX_OUTPUT_TOKENS

```
ChatGoogleGenerativeAI(model="gemini-2.5-pro",
temperature=0.5,max_output_tokens=MAX_OUTPUT_TOKENS)
```

- CHUNK_SIZE: how many chars in each chunk
- CHUNK_OVERLAP: how many chars consecutive chunks can overlap so sentences aren't cut in half
- MAX_CHUNKS_PER_FILE: how many chunks keep in each file, actually not relevant unless chunks don't cover the whole file
- MAX_CONTEXT_CHUNKS：after combining all retrieved chunks (across files), how many chunks send to LLM at most
- TOP_K_PER_FILE, MMR_FETCH_K_MULT, MMR_FETCH_K_MIN, MMR_LAMBDA_MULT:

```
fetch_k = max(MMR_FETCH_K_MIN, TOP_K_PER_FILE * MMR_FETCH_K_MULT)
```

```
retriever = index_by_file[fn].as_retriever(
            search_type="mmr",
            search_kwargs={"k": TOP_K_PER_FILE, "fetch_k": fetch_k,
"lambda_mult": MMR_LAMBDA_MULT}
        )
```

For MMR_LAMBDA_MULT, higher (closer to 1) = more relevance-focused, less diversity, lower (closer to 0) = more diverse chunks, less strict relevance

3.2 Iteration and Refinement:

The comparison table below displays the evolution of the system across multiple iterative versions, starting from the baseline implementation provided by the instructor and progressing through versions v1 to v4. Each version introduced specific improvements to the RAG pipeline, including retrieval strategies, citation mechanisms, scope control for multi-document retrieval, and system stability fixes.

The comparison highlights how the system gradually evolved from a basic single-index chatbot into a more robust architecture with balanced multi-file retrieval, reproducible evidence display, and configurable conversational behavior. This iterative development process allowed us to

address practical engineering challenges such as cross-document retrieval bias, embedding API limits, and Streamlit state management issues.

Comparison Table for Iterative Versions:

Version	Major changes vs Caseline	RAG Config (chunk/retrieval)	Citation/Evidence	Scope / Multi-file	Stability fixes
Baseline `chat_with_pdf_gemini_with_history.py`	Single FAISS store for all PDFs; basic UI	chunk 500 / overlap 50; MMR k=4 fetch_k=20	Shows retrieved chunks in expander; no filename/page/section citations (source only page sometimes)	No scope control; mixed-doc bias likely	API key handling incorrect; multi-file bias not addressed
v1	Adds TOC parsing + section metadata; ConversationalRetrievalChain; formatting-safe streaming	chunk 800 / overlap 100	Sources footer: filename/page/section	Still single combined store (bias can remain)	Fixes QA chain input validation errors caused by missing prompt variable, better streaming formatting
v2	Per-file FAISS + disk cache; scope selection; balanced per-file retrieval; embedding retry	chunk 1500 / overlap 150; TOP_K_PER_FILE=4 (top 4 most relevant chunks); MAX_CONTEXT=12 (the best 12 chunks get passed to the LLM)	Inline citations `[i]` aligned to context; sources footer aligned	Sidebar scope selection; avoids single-doc dominance	Handle Gemini embedding API rate-limit errors (HTTP 429); Introduce local disk cache of of FAISS indexes; reduces rebuild cost
v3	Adds Evidence panel with excerpt + full chunk dropdown	same as v2	Evidence expander shows retrieved chunks for reproducibility	Same as v2	Better auditability; still limited toggles
v4	Adds QA toggles: history on/off; history turns slider; evidence cited-only toggle; preserves evidence on rerender	same as v2/v3 but configurable history	Stores evidence in message state; toggling doesn't lose evidence	Scope buttons use callbacks to avoid session_state errors	Fixes Streamlit widget-state errors; improves UX robustness

The reason we iterated the system through several versions is that a number of issues and biases were observed during development and testing. These problems include retrieval bias across multiple documents, embedding API rate limits, and Streamlit session state errors. The following case studies illustrate these issues and how they were addressed through system design improvements.

Case 1: Multi-document Retrieval Bias

During early experiments, we observed a retrieval bias when multiple 10-K reports were uploaded simultaneously. For example, when both Amazon and Microsoft reports were indexed, asking:

"What was the total cash flow of Amazon in 2024?"

This bias sometimes produced answers containing Microsoft's financial values. This occurred because the retriever returned mixed chunks from multiple documents, and the LLM generated answers based on the most dominant retrieved context. To mitigate this issue, we introduced a document scope selection mechanism in the user interface. When the question does not clearly indicate the target document, the chatbot asks the user to select which document(s) should be used for retrieval, thereby reducing cross-document bias.

Case 2: Streamlit Session State Error

During development of the Streamlit interface, we encountered several runtime errors related to Streamlit's session state management. For example, when switching the retrieval scope from a single document to "Use all documents", the system sometimes produced the error:

"`StreamlitAPIException: session_state cannot be modified after the widget with the same key is instantiated.`"

This error occurred because the application attempted to modify `st.session_state` variables after the corresponding UI widget had already been created in the same execution cycle. To resolve this issue, we redesigned the UI logic by introducing callback-based state updates using `on_click` functions and carefully controlling the order of widget initialization.

This ensures that session state variables are updated before the widgets are instantiated, preventing runtime conflicts. As a result, the interface became more stable when users dynamically adjusted document scope or other configuration settings.

Case 3: Reproducible Evidence

During early development, the chatbot generated answers with citations that referenced document pages and sections, but users could not directly verify the supporting evidence without manually locating the text in the original PDF. This reduced the transparency and reproducibility of the generated answers. For example, the chatbot might output:

`"Amazon faces regulatory risks in China and India [1]."`

Even though the system listed the corresponding page and section in the source list, users still had difficulty confirming the exact text supporting the claim. To improve transparency, we introduced an Evidence panel that displays the retrieved text chunks used during generation. Each citation index `[i]` is linked to an excerpt from the original document, and users can expand the panel to view the full retrieved chunk. This displayed evidence mechanism allows readers to directly inspect the evidence supporting each answer, improving credibility and reproducibility.

Case 4: Retrieval Context Fragmentation

In early versions of the system, a relatively small chunk size (e.g., 500 tokens with 50-token overlap) was used during document splitting. However, financial reports such as 10-K filings often contain long paragraphs and structured explanations. Hence, important information was sometimes split across multiple chunks, causing the retriever to return incomplete context.

For example, a financial explanation describing operating cash flow might span several paragraphs, but only one partial chunk would be retrieved, leading to incomplete or misleading answers. To address this issue, we increased the chunk size and overlap parameters (e.g., 1500 tokens with 150-token overlap). This adjustment preserved more contextual information within each chunk and improved the quality of retrieved passages, resulting in more accurate and complete answers.

4. Parameter tuning and evaluation

After the above refinement, we conducted hyperparameter tuning and evaluation across multiple RAG configurations (A–S) to analyze the impact of chunking, retrieval depth, and MMR parameters on system performance.

The parameter combinations are organized into four stages.

A–F were designed to explore a wide range of parameter settings.

G–M were developed based on configuration F, which showed the best performance among A–F.

N–P further refine the configurations by building around F and M, the two best-performing settings identified in previous experiments.

Q–S were designed to test more aggressive retrieval settings in order to handle particularly challenging questions.

Baseline is our given chat_with_pdf_gemini_with_history.RAG.

Config	MAX_OUTPUT_TOKENS	CHUNK_SIZE	CHUNK_OVERLAP	MAX_CHUNKS_PER_FILE	TOP_K_PER_FILE	MAX_CONTEXT_CHUNKS	MMR_FETCH_K_MULT	MMR_FETCH_K_MIN	MMR_LAMBDA_MULT
A	1024	1800	120	250	2	4	4	20	0.65
B	1536	1400	150	600	3	8	5	30	0.60
C	2048	1100	160	1000	4	12	6	40	0.55
D	2048	800	200	2200	7	22	8	60	0.45
E	2048	1200	120	1200	2	6	5	30	0.72
F	2048	1000	140	1400	5	14	7	50	0.50
G	1536	1100	120	1400	3	10	6	48	0.70
H	2048	1000	140	1600	5	14	10	80	0.52
I	2048	950	160	1600	6	16	8	60	0.40
J	2048	750	180	2200	7	18	8	70	0.48
K	2048	1400	140	1200	4	12	7	56	0.58
L	1536	1000	120	1400	4	12	9	72	0.55
M	2048	1500	150	1500	6	18	10	80	0.50
N	2048	1000	140	1600	7	20	10	80	0.50
O	2048	1200	160	1600	6	16	12	100	0.50

P	2048	1400	180	1600	6	18	10	80	0.55
Q	2048	1000	140	1800	7	20	12	100	0.48
R	2048	1200	160	1800	7	20	12	100	0.50
S	2048	1500	180	1800	8	24	14	120	0.45

Evaluation of output are based on designed 54 questions with expected answers conducted in 2 parts:

1. Semantic Similarity Evaluation (BERT)

   First, the semantic similarity between the model output and a reference answer was computed using a BERT-based embedding model. Both the generated response and the designed reference answer were embedded into vector representations, and their cosine similarity was calculated.

   If a non adversarial question is answered by a refusal word, the similarity is 0.

2. LLM-Based Qualitative Evaluation

   After selecting 5 best performing parameter groups from the former step, in addition to semantic similarity, responses were evaluated using a GPT-based grading framework. For each question, the model output and the associated citations were sent to GPT, which acted as a strict evaluator.

   GPT assessed the responses across four dimensions:

   1) Hallucination: Whether the response introduced information that was not supported by the retrieved documents.
   2) Accuracy: Whether the factual statements in the response were correct and consistent with the source documents.
   3) Relevance: Whether the response directly addressed the question and remained focused on the requested information.
   4) Retrieval Quality: Whether the cited passages were appropriate and contained the evidence needed to support the answer.

Each dimension was scored on a fixed scale 0-5, allowing consistent comparison across different system configurations.

Score	Hallucination	Accuracy	Relevance	Retrieval Quality
5	No hallucination. All statements consistent with the 10-K filings.	Fully correct answer with correct values or explanation.	Directly answers the question clearly.	Correct section/page retrieved and clearly supports the answer.
4	Mostly correct with only minor wording differences or unverifiable paraphrasing.	Mostly correct with small rounding or wording differences.	Mostly relevant but includes minor extra information.	Correct document retrieved but citation is slightly vague.
3	Some unsupported details appear but core facts remain plausible.	Partially correct; key elements missing or slightly inaccurate.	Partially answers the question but incomplete.	Relevant document retrieved but evidence is not precise.
2	Significant unsupported claims appear.	Mostly incorrect but contains small correct fragments.	Weakly related to the question.	Weak retrieval with only loosely related sources.
1	Major hallucinations or fabricated key facts.	Largely incorrect answer.	Barely related to the question.	Generic answer without meaningful retrieval evidence.
0	Completely fabricated answer or entirely unsupported by the documents.	No answer or completely incorrect response.	Completely unrelated response.	Retrieval clearly incorrect or irrelevant.

For adversarial questions and refusal:

Case	Score behavior
Adversarial + correct refusal	hallucination=5 accuracy=5 relevance=5 retrieval=5
Adversarial + refusal phrase	hallucination=1 accuracy=1 relevance=1 retrieval=1
Non-adversarial + refusal	hallucination=5 accuracy=0 relevance=1 retrieval=1

4.1 Question Design and Expected Output

The questions set was deliberately constructed as a comprehensive testing framework for evaluating the model, spanning four critical dimensions: financial performance, business strategy, risk assessment, and operational metrics. In each dimension, the questions are constructed within a progressive range of difficulty, from simple fixed answers to comprehensive comparative analysis questions. Simple questions test whether the model can locate specific numbers or facts at a single document location. Advanced questions require multi-step reasoning,

such as calculating changes during the reporting period, identifying the main drivers of financial trends, and spanning multiple pages and integrating information. Moreover, adversarial questions further push the boundaries by asking for information not included in the file, testing whether the model can recognize the limitations of its knowledge base instead of generating hallucinatory responses. Finally, the challenge problem requires cross-document synthesis and implicit reasoning, demanding that the model compare the disclosures of three different companies and form judgment-based conclusions regarding strategic priorities, accounting estimates, and legal threats. The structure of the questions ensures that the question set tests not only retrieval accuracy but also reasoning depth, boundary recognition, and comparative analysis capabilities.

The expected answers were derived systematically from the source 10-K files with precise citations, establishing a verifiable ground truth for each question. For quantitative items, answers were verified against the financial statements to ensure numerical accuracy. For analytical questions, expected answers were constructed by tracing the logics of evidence presented in the different sections of files, ensuring that sample answer follows directly from documented information. To validate the completeness and appropriateness of this answer key, we also employed mature research chatbot systems including NotebookLM as independent evaluators. The system was provided with the same source documents and questions, and their responses were compared and used for accuracy, citation quality, and reasoning coherence. This validation methodology transforms the expected answers into a reliable reference standard against which model performance can be meaningfully measured.

4.2 Evaluation (Table)

In semantic similarity evaluation, here is the result. All adversarial questions are replied with refusal words. Base RAG doesn’t support a refusal word for adversarial questions so it tries to give some information even when there isn’t relevant information in files.

	Easy	Advanced	Challenge	Financial	Business	Risk	Performance	Overall	Adversarial_answered	Non-Adv. not answered	Non Adv. partially answered

a	0.329249	0.196118	0.000000	0.261623	0.196098	0.257556	0.286336	0.215980	0	31	0
b	0.564462	0.514203	0.311157	0.534965	0.609197	0.480995	0.469105	0.505575	0	11	2
c	0.570548	0.770986	0.483535	0.647486	0.679602	0.692042	0.802253	0.668242	0	2	2
d	0.556177	0.753855	0.480987	0.637402	0.696874	0.709616	0.692003	0.653854	0	4	2
e	0.521999	0.714162	0.237126	0.549295	0.664554	0.700321	0.706604	0.590478	0	7	0
f	0.553163	0.756637	0.581825	0.651461	0.683579	0.703751	0.696219	0.666961	0	2	1
g	0.546597	0.517799	0.318477	0.510415	0.490603	0.484712	0.662197	0.502483	0	12	1
h	0.570653	0.718983	0.267419	0.596427	0.688411	0.709298	0.692337	0.613094	0	6	0
i	0.547752	0.783242	0.376390	0.624735	0.689387	0.699644	0.814029	0.653889	0	4	2
j	0.579673	0.763263	0.217351	0.620865	0.689206	0.712412	0.807218	0.633828	0	5	0
k	0.538885	0.731512	0.222617	0.610906	0.667454	0.586172	0.798746	0.603691	0	6	0
l	0.571428	0.618747	0.285717	0.544764	0.680940	0.484209	0.694828	0.561345	0	9	1
m	0.535227	0.773646	0.404988	0.623955	0.678098	0.711723	0.764012	0.648091	0	3	0
n	0.549205	0.788679	0.591047	0.667616	0.658116	0.698953	0.807161	0.684150	0	2	1
o	0.561725	0.756117	0.226006	0.593986	0.688283	0.712536	0.796394	0.625056	0	4	0
p	0.568805	0.749225	0.371060	0.647982	0.616312	0.704264	0.809890	0.641814	0	4	2
r	0.584971	0.717605	0.606709	0.618201	0.674661	0.616974	0.791342	0.659532	0	3	1
s	0.569881	0.764284	0.372062	0.641027	0.676646	0.689610	0.797306	0.650455	0	3	1
base	0.535367	0.689389	0.658408	0.591447	0.632755	0.612534	0.714548	0.634176	0	0	11

None of the values reaches overfitting and they shouldn't, so the higher the score the better. N is the best so far and we select 5 parameter groups to LLM-Based Qualitative Evaluation. Here are the scores in each dimension:

1) Hallucination

	Easy	Advanced	Challenge	Financial	Business	Risk	Performance	Overall
base	4.1250	4.461538	4.666667	4.142857	4.250000	4.500	4.625	4.000000
d	4.1250	4.038462	4.500000	4.142857	4.000000	4.000	4.125	3.777778
i	4.0625	4.038462	4.666667	4.071429	4.083333	4.000	4.000	3.777778
l	4.0000	4.192308	4.833333	4.142857	4.000000	4.250	4.125	3.851852
n	4.0625	4.000000	4.333333	4.000000	4.083333	4.000	4.000	3.722222
r	4.0000	4.076923	4.333333	4.071429	4.000000	4.125	4.000	3.740741

2) Accuracy

	Easy	Advanced	Challenge	Financial	Business	Risk	Performance	Overall
base	3.50	2.153846	1.333333	3.428571	3.000000	2.0	1.5	2.333333
d	3.50	3.846154	2.000000	3.428571	4.000000	4.0	3.5	3.222222
i	3.75	3.846154	1.333333	3.714286	3.666667	4.0	4.0	3.222222
l	4.00	3.230769	0.666667	3.428571	4.000000	3.0	3.5	2.925926
n	3.75	4.000000	2.666667	4.000000	3.666667	4.0	4.0	3.444444
r	4.00	3.692308	2.666667	3.714286	4.000000	3.5	4.0	3.370370

3) Relevance

	Easy	Advanced	Challenge	Financial	Business	Risk	Performance	Overall
base	4.50	3.153846	2.333333	4.428571	4.000000	3.0	2.5	3.222222
d	4.50	4.846154	3.000000	4.428571	5.000000	5.0	4.5	4.111111
i	4.75	4.846154	2.333333	4.714286	4.666667	5.0	5.0	4.111111
l	5.00	4.230769	1.666667	4.428571	5.000000	4.0	4.5	3.814815
n	4.75	5.000000	3.666667	5.000000	4.666667	5.0	5.0	4.333333

r	5.00	4.692308	3.666667	4.714286	5.000000	4.5	5.0	4.259259

4) Retrieval Quality. Because base doesn't support citations, it is not included

	Easy	Advanced	Challenge	Financial	Business	Risk	Performance	Overall
d	3.6250	3.884615	2.5	3.571429	4.00	4.000	3.625	3.333333
i	3.8125	3.884615	2.0	3.785714	3.75	4.000	4.000	3.333333
l	4.0000	3.423077	1.5	3.571429	4.00	3.250	3.625	3.111111
n	3.8125	4.000000	3.0	4.000000	3.75	4.000	4.000	3.500000
r	4.0000	3.769231	3.0	3.785714	4.00	3.625	4.000	3.444444

In conclusion, combination N wins in almost all dimensions, so that is our final combination. Q53 is supposed to be partially answered. Apart from that, our best combination N fails to ask 2 questions. They are:

- Q18. How does Alphabet generate most of its revenue?
  This question is categorized as Easy, and the system should normally answer it correctly. We tried this question again and it was answered. However, the LLM occasionally fails to generate the correct response. This behavior occurs intermittently across multiple runs even with the best configuration, so we consider our data reliable with these failures. We attribute this to LLM variability rather than a retrieval issue.
- Q49. Using only the financial data from the three 10-K filings, determine which company has the highest return on equity (ROE) and discuss what structural factors might make this comparison misleading.

  The question requires locating net income and shareholders' equity across three separate filings, performing three calculations, identifying the highest value, and then critically analyzing why the comparison may be distorted by factors like goodwill inflation, excess cash holdings, or capital-intensive business models. Our model retrieved partial financial data but could not consistently execute the full calculation sequence while simultaneously identifying the nuanced structural factors that undermine direct comparability.

We tested various parameter combinations. Each configuration retrieved relevant context but failed to generate an accurate and complete answer. We conclude that further adjustments to the model will only result in diminishing returns, as the ability to synthesize across documents for these problems exceeds the capabilities of our current model architecture. This question highlights the limitations of the model's capabilities when the question structure requires complex cross-document synthesis and calculation.

In conclusion, we consider this a good state to stop tuning and accept parameter combinations N.

4.3 Further Steps

During evaluation, we observed limitations when the system solved questions requiring numerical calculations and multi-step reasoning. For example, one question required computing a financial metric based on values reported in the 10-K filings. Although the system was able to retrieve relevant passages containing the financial numbers, the generated answer did not always produce the correct calculation.

This issue reflects several practical bottlenecks in RAG-based systems: (1) the reliability of LLM-based mathematical computation, (2) multi-step reasoning across multiple retrieved contexts, and (3) the extraction of structured financial information from tables in long reports. Our system primarily focuses on retrieving relevant document chunks and providing them as context to the language model, while reasoning and calculation are handled by the underlying LLM (Google Gemini). Hence, the correctness of computation-heavy questions depends largely on the model’s reasoning capabilities rather than the retrieval system.

Future improvements could address this limitation by integrating tool-assisted calculations, structured financial table extraction, or prompting strategies that guide the model through intermediate reasoning steps, which may improve the reliability of complex financial analysis tasks.

5. Contribution of Team Members

Group 1 Team members: Qi Kan, Tianjin Duan, Xiaojia Wang, Yixuan Han, Zihan Wang, Calis Nguyen, Emanuel Telles Chaves.

We separated the work into 3 groups: Group A, Group B, and Group C.

Here is our Teamwork Contribution and Workflow (A → B → C):

1. Group A: Question Bank Creation
    - Emanuel Telles Chaves, Qi Kan
2. Group B: RAG Pipeline, Evaluation, and Persona
    - Yixuan Han, Tianjin Duan, Calis Nguyen
3. Group C: Presentation Slides
    - Xiaojia Wang, Zihan Wang

Process Flow:

- Group A → Develops question bank → Group B
- Group B → Builds/evaluates RAG pipeline and documents findings → Group C
- Group C → Designs slides based on Group A and Group B's outputs for final presentation.

.